International Institute of Information Technology, Bangalore

Department of Computer Science and Engineering

CS 816 – Software Production Engineering

Mini Project Report

CI/CD Pipeline Implementation for a Scientific Calculator Application

Submitted By:

Vivek Joshi

Roll No: MT2025133

Submitted To:

Dr. B. Thangaraju

GitHub Repository:

`https://github.com/vivekvj18/SPE-Scientific-Calculator`

Docker Hub Image:

`https://hub.docker.com/repository/docker/2001vivekjoshi/calculator-app/general`

Date of Submission:

March 6, 2026

Contents

1 Introduction **3**

2 Project Architecture **3**

3 Application Overview **3**
3.1 Supported Operations 4

4 Unit Testing **4**
4.1 Test Coverage Summary 4

5 CI/CD Pipeline Implementation **5**
5.1 Pipeline Stages Overview 5
5.2 Stage-by-Stage Explanation 6

6 Configuration Files **8**
6.1 Dockerfile 8
6.2 Maven Build File (pom.xml) 8
6.3 Ansible Inventory File 8
6.4 Ansible Deployment Playbook 9

7 Results and Observations **9**

8 Conclusion **9**

1 Introduction

Continuous Integration and Continuous Deployment (CI/CD) are foundational DevOps practices that streamline the software development lifecycle. CI ensures that code changes are frequently integrated and automatically validated through tests. CD automates the packaging and deployment process, enabling faster and more reliable software releases.

In this project, a command-line Scientific Calculator developed in Java serves as the target application. The application is integrated with a Jenkins pipeline that automates building, unit testing, containerisation with Docker, image publishing to Docker Hub, and deployment via Ansible. Post-build email notifications are also configured to alert the developer of pipeline outcomes.

2 Project Architecture

The end-to-end pipeline workflow is as follows:

GitHub → Jenkins → Docker Hub → Ansible

Push	Build / Test	Push Image	Deploy Container

When a developer pushes code to the GitHub repository, Jenkins automatically triggers the pipeline. The pipeline compiles the application, runs all unit tests, builds and pushes a Docker image, and finally deploys a fresh container via an Ansible playbook.

Technologies used:

- Java 21 — Application language
- Maven — Build tool and dependency management
- JUnit 5 — Unit testing framework
- Jenkins — CI/CD automation server
- Docker — Application containerisation
- Docker Hub — Container image registry
- Ansible — Automated deployment orchestration
- Git / GitHub — Version control and source repository

3 Application Overview

The Scientific Calculator is a command-line application written in Java. It consists of two main classes: `Calculator.java`, which implements the mathematical operations, and `Main.java`, which provides the interactive menu-driven interface.

3.1 Supported Operations

Option	Operation	Method	Notes
1	Addition	`add(a, b)`	Returns $a + b$
2	Subtraction	`subtract(a, b)`	Returns $a - b$
3	Multiplication	`multiply(a, b)`	Returns $a \times b$
4	Division	`divide(a, b)`	Throws on $b = 0$
5	Power	`power(a, b)`	Uses `Math.pow`
6	Square Root	`sqrt(a)`	Throws on $a < 0$
7	Factorial	`factorial(n)`	Throws on $n < 0$
8	Natural Logarithm	`ln(a)`	Throws on $a \leq 0$

The `Calculator` class implements each operation as a separate method with built-in input validation. Invalid inputs such as division by zero, negative square roots, or non-positive logarithm arguments throw appropriate Java exceptions, which are caught and displayed to the user in the menu interface.

4 Unit Testing

Unit tests are written using JUnit 5 and reside in `CalculatorTest.java`. The tests cover both expected outputs and exception-throwing edge cases for every operation.

4.1 Test Coverage Summary

Test Method	Cases Covered	Edge Cases
`testAdd()`	Positive, negative, zero	—
`testSubtract()`	Positive, negative, zero	—
`testMultiply()`	Positive, negative, zero	—
`testDivide()`	Positive, negative	Division by zero
`testPower()`	Integer, zero, negative exp.	—
`testSquareRoot()`	Perfect square, zero	Negative input
`testLogarithm()`	Value > 1, value $= 1$	Input $= 0$
`testFactorial()`	$n = 5$, $n = 0$, $n = 1$	Negative input

Each test method uses `assertEquals` with a delta of `0.0001` for floating-point comparisons, and `assertThrows` to verify that illegal inputs correctly raise exceptions. Tests are executed automatically during the Test stage of the Jenkins pipeline via `mvn test`.

5 CI/CD Pipeline Implementation

The CI/CD pipeline is defined in a `Jenkinsfile` at the root of the repository. Jenkins reads this file and executes each stage sequentially. Figure 1 shows the complete pipeline as viewed in Jenkins after a successful run.

5.1 Pipeline Stages Overview

Stage	Name	Description
1	Build	Compile source code and package JAR with Maven
2	Test	Run all JUnit 5 unit tests
3	Docker Build & Tag	Build and tag the Docker image
4	Docker Push	Authenticate and push image to Docker Hub
5	Deploy using Ansible	Pull image and redeploy container via playbook

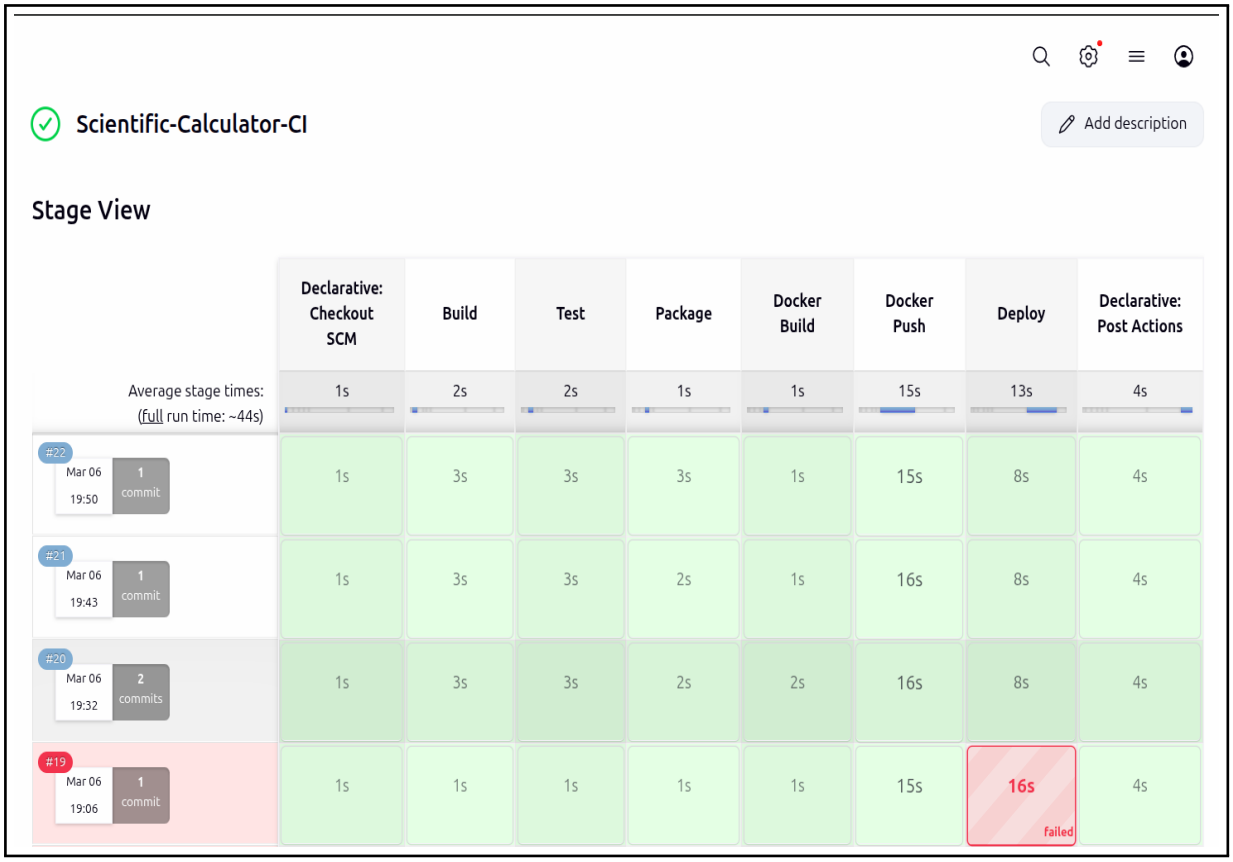

Figure 1: Jenkins pipeline view showing all five stages completing successfully

5.2 Stage-by-Stage Explanation

Stage 1 — Build

The Build stage compiles the Java source code and packages it into an executable JAR using Maven. The `clean` goal removes any previously compiled `target/` artifacts to ensure a fresh build. The `package` goal compiles the source and produces `scientific-calculator-1.0-SNAPSHOT.jar` in the `target/` directory. The `-DskipTests` flag explicitly skips test execution during this stage — testing is intentionally delegated to the dedicated Test stage that follows. This separation ensures that the Build stage is responsible solely for compilation and packaging, while the Test stage owns all quality validation.

Stage 2 — Test

The Test stage executes all JUnit 5 unit tests via the Maven Surefire plugin. If any test fails, Maven exits with a non-zero status code, which causes Jenkins to immediately mark the pipeline as **FAILED** and skip all remaining stages. This acts as a quality gate — no Docker image is ever built or deployed from broken code.

Stage 3 — Docker Build & Tag

Once the application has passed all tests, a Docker image is built from the `Dockerfile` at the project root. The first command builds the image and tags it locally as `calculator-app:1.0`. The second command re-tags it with the full Docker Hub repository path so it is ready to be pushed in the next stage. Combining both operations in a single stage keeps image creation and naming as one atomic step.

Stage 4 — Docker Push

The tagged image is pushed to Docker Hub using credentials stored securely in Jenkins. The `withCredentials` block injects the Docker Hub username and password from the Jenkins credential store as environment variables. Passing the password via `--password-stdin` prevents the secret from appearing in process listings or build logs. Once pushed, the image is publicly accessible on Docker Hub for deployment on any host.

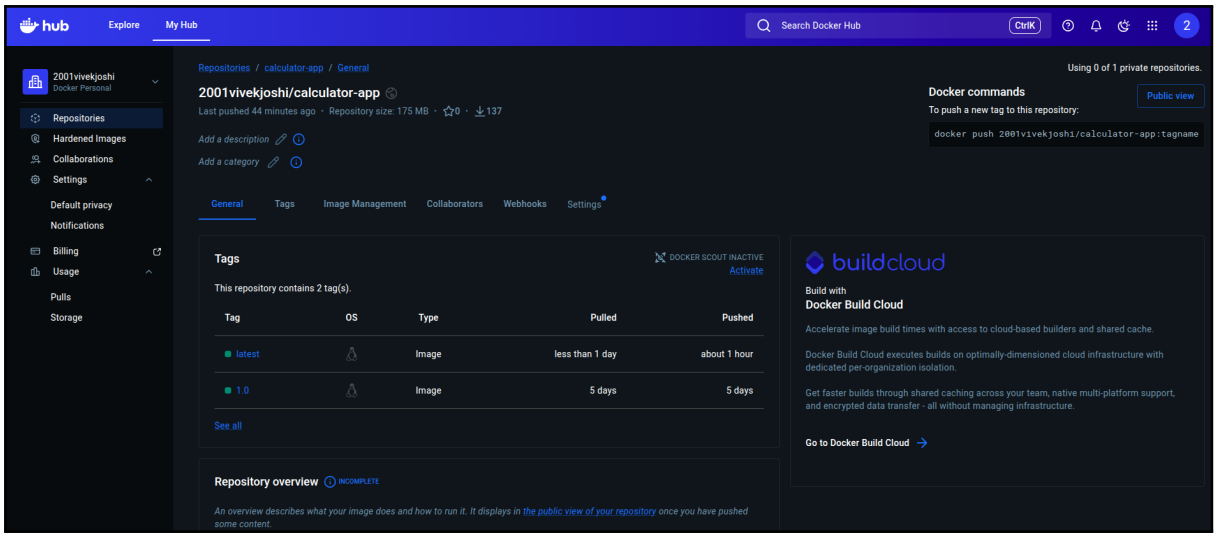

Figure 2: Docker Hub showing the published calculator-app:1.0 image

Stage 5 — Deploy using Ansible

The final stage triggers the Ansible playbook to deploy the updated container. The playbook connects to the target host, pulls the latest image from Docker Hub, stops and removes the previously running container, and starts a fresh one. This ensures zero stale state from previous deployments. The use of Ansible makes the deployment step declarative and repeatable across different environments without modifying the `Jenkinsfile`.

Post Block — Email Notifications

The `post` block runs after all stages complete, regardless of outcome. It uses the `emailext` Jenkins plugin to send an email to all developers who committed code in that build. The email subject clearly states **SUCCESS** or **FAILED** along with the build number, and the body includes a direct link to the Jenkins build URL for quick inspection. This closes the developer feedback loop without requiring anyone to monitor the Jenkins dashboard manually.

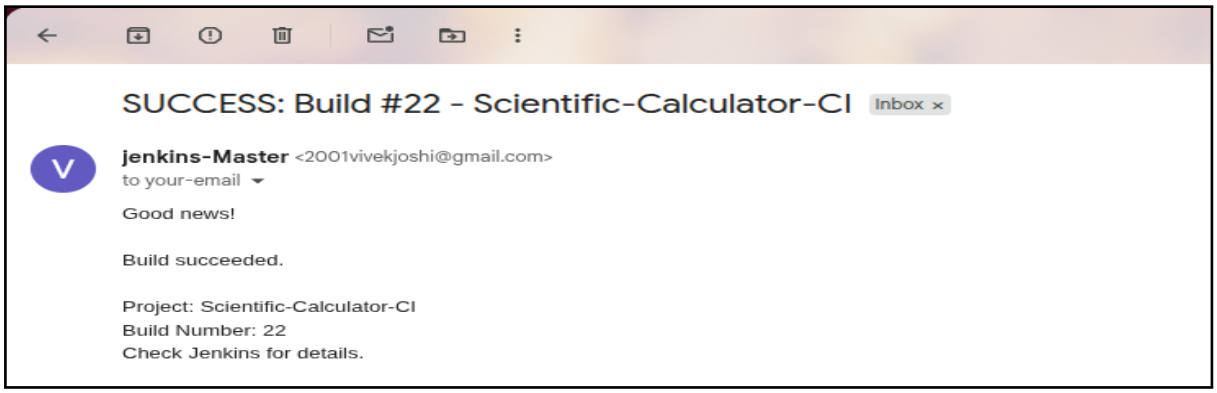

Figure 3: Success notification email received after all five pipeline stages passed

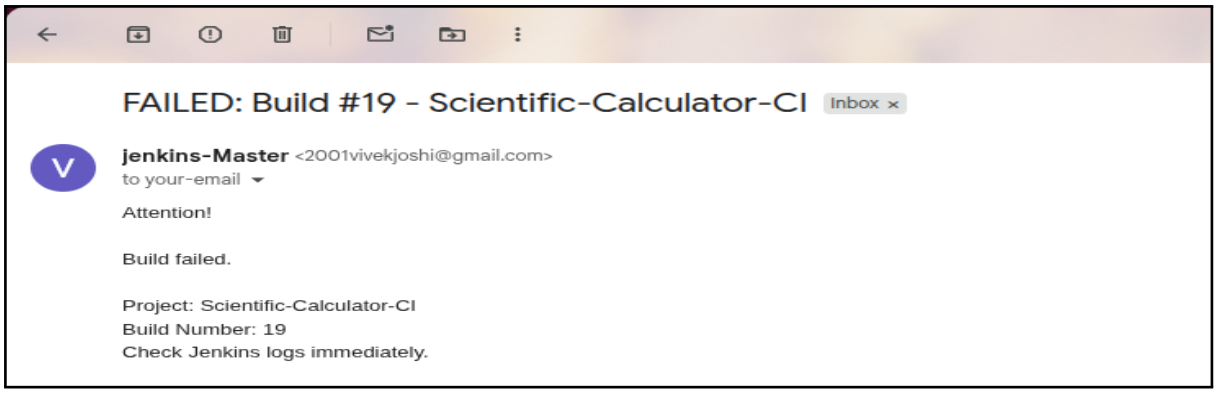

Figure 4: Failure notification email received when a pipeline stage did not complete successfully

6 Configuration Files

6.1 Dockerfile

The Dockerfile packages the compiled JAR into a lightweight Alpine-based container image using the Eclipse Temurin JRE 21.

Design decisions:

- `eclipse-temurin:21-jre-alpine` is chosen over a full JDK image because only the Java Runtime Environment is needed to run the pre-compiled JAR, keeping the image smaller and more secure.
- The working directory is set to `/app`, the JAR is copied in, and the container is started via `CMD ["java", "-jar", "app.jar"]`.

6.2 Maven Build File (pom.xml)

The project uses Maven with Java 21. Key dependencies and plugins are:

- **JUnit Jupiter 5.10.0** — Unit testing (test scope only)
- **maven-compiler-plugin 3.11.0** — Compiles at Java 21 release level
- **maven-surefire-plugin 3.2.5** — JUnit 5 compatible test runner
- **maven-jar-plugin 3.3.0** — Packages JAR with `org.vivek.Main` as the entry point

6.3 Ansible Inventory File

- `[myhosts]` — Defines the target host group used by the playbook.
- `ansible_connection=local` — Tells Ansible to run tasks directly on the Jenkins host without SSH.
- `ansible_python_interpreter` — Explicitly sets the Python 3 interpreter to avoid ambiguity on systems with multiple Python installations.

6.4 Ansible Deployment Playbook

The `deploy.yml` playbook performs three sequential tasks:

1. **Pull latest Docker image** — Fetches the tagged image from Docker Hub using the `community.docker` Ansible collection.
2. **Stop existing container** — Removes any running container named `calculator-container` (state: absent) to avoid port conflicts or stale processes.
3. **Run new container** — Starts a fresh container. `tty: true` and `interactive: true` are required because the calculator is a terminal-interactive application that reads from standard input.

7 Results and Observations

The CI/CD pipeline was implemented and validated successfully. The following outcomes were observed:

- **Automated builds:** Every push to the GitHub repository triggers the full pipeline automatically, eliminating the need for manual compilation or packaging.
- **Test reliability:** All 8 unit tests pass consistently, covering both normal inputs and boundary/exception cases for every calculator operation.
- **Secure credential management:** Docker Hub credentials are injected at runtime via Jenkins' credential store, preventing secrets from appearing in logs or source code.
- **Zero-downtime redeploy:** The Ansible playbook stops the old container before starting a new one, ensuring the latest version is always running with no residual state from previous builds.
- **Developer feedback loop:** Email notifications via the `emailext` plugin provide instant feedback on build outcomes without requiring the developer to monitor the Jenkins dashboard.

8 Conclusion

This project demonstrates a complete CI/CD pipeline integrating Jenkins, Docker, and Ansible to automate the software delivery process for a Java application. The pipeline handles every step from source code compilation to live container deployment without manual intervention.

Key engineering decisions — such as using a lightweight JRE Alpine image, securing credentials through Jenkins' credential store, and enabling interactive TTY for a terminal-based application — reflect practical considerations that arise when deploying real applications in automated environments.

Such pipelines improve development velocity, reduce human error in deployments, and provide a reproducible and auditable release process. Future improvements may include integrating monitoring tools, deploying the application using container orchestration platforms such as Kubernetes, and implementing automated rollback mechanisms.